

KUBERNETES – 01

=====

1. Setup Minikube in your local machine.

◆ **Step 1: Verify Docker Installation**

checked Docker version:

`docker --version`

Output:

Docker version 28.5.1

Docker is installed and working
(Minikube will use Docker as the driver)

◆ **Step 2: Create Minikube Directory**

created a folder in C drive:

`New-Item -Path "C:\" -Name "minikube" -ItemType Directory -Force`

Directory created:

C:\minikube

◆ **Step 3: Download Minikube Binary**

downloaded minikube.exe:

```
$ProgressPreference = 'SilentlyContinue'; Invoke-WebRequest -OutFile 'c:\minikube\minikube.exe' -Uri  
'https://github.com/kubernetes/minikube/releases/latest/download/minikube-windows-amd64.exe' -  
UseBasicParsing
```

Now:

C:\minikube\minikube.exe

◆ **Step 4: Add Minikube to System PATH**

updated environment variable:

```
$UserPath = [Environment]::GetEnvironmentVariable('Path', [EnvironmentVariableTarget]::User)
```

```
[Environment]::SetEnvironmentVariable(  
    'Path',  
    "$UserPath;C:\minikube",  
    [EnvironmentVariableTarget]::User  
)
```

This allows you to run minikube from anywhere.

◆ Step 5: Verify Minikube Installation

In Git Bash / PowerShell:

minikube version

Output:

minikube version: v1.38.0

Minikube installed successfully

◆ Step 6: Start Minikube Cluster

started Minikube using Docker driver:

minikube start --driver=docker

- Using container runtime: **containerd**
- Pulled Kubernetes base image
- Created control-plane node
- Configured CNI
- Enabled addons:
 - storage-provisioner
 - default-storageclass

Cluster successfully started

◆ Step 7: Verify Minikube Cluster

Check cluster IP:

minikube ip

Output:

192.168.49.2

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\SRIHARI> New-Item -Path 'c:\' -Name 'minikube' -ItemType Directory -Force

    Directory: C:\

Mode                LastWriteTime         Length Name
----                -
d-----          9/21/2026   9:10 PM             minikube

PS C:\Users\SRIHARI> $ProgressPreference = 'SilentlyContinue'; Invoke-WebRequest -OutFile 'c:\minikube\minikube.exe' -Uri 'https://github.com/kubernetes/minikube/releases/latest/download/minikube-windows-amd64.exe' -UseBasicParsing

PS C:\Users\SRIHARI> $userPath = [Environment]::GetEnvironmentVariable('Path', [EnvironmentVariableTarget]::User)
PS C:\Users\SRIHARI> [Environment]::SetEnvironmentVariable(
>>     'Path',
>>     "$userPath;c:\minikube",
>>     [EnvironmentVariableTarget]::User
>> )
```

```

PS C:\Users\SRIHARI> minikube version
minikube version: v1.39.0
commit: 7a9f6a841470a207de8cf4bafcccee0969d8ba10
PS C:\Users\SRIHARI> minikube start --driver=docker
* minikube v1.39.0 on Microsoft Windows 11 Home Single Language 25H2
* Using the docker driver based on user configuration
* Using Docker Desktop driver with root privileges
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.51 ...
* Downloading Kubernetes v1.37.0 preload ...
  > preloaded-images-k8s-v18-v1...: 347.58 MiB / 347.58 MiB 100.00% 6.18 Mi
  > gcr.io/k8s-minikube/kicbase: 507.61 MiB / 507.61 MiB 100.00% 5.52 MiB p
* Creating docker container (CPUs=2, Memory=3072MB) ...
* Preparing Kubernetes v1.37.0 on containerd 2.3.4 ...
* Configuring CNI (Container Networking Interface) ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass

! C:\Program Files\Docker\Docker\resources\bin\kubectl.exe is version 1.34.1, which may have incompatibilities with Kubernetes 1.37.0.
  - Want kubectl v1.37.0? Try 'minikube kubectl -- get pods -A'
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
PS C:\Users\SRIHARI> minikube ip
192.168.49.2

```

1. Setup k8s master and two worker nodes on ubuntu.

Create instances first, then use their private IPs

This is the easiest approach.

1. Create Master.
2. Create Worker01.
3. Create Worker02.

EC2 → Launch instance → Network settings → Edit → Private IP

Choose:

Private IPv4 address → Manual

Then enter an available IP from your subnet.

⚠ Make sure the IP is inside the subnet's CIDR and is not already being used.

Now we know your subnet:

VPC: 172.31.0.0/16

Subnet: 172.31.4.0/22

AZ: us-east-1a

Your subnet IP range

172.31.4.0/22 covers:

172.31.4.0



172.31.7.255

So you can choose private IPs such as:

Master → 172.31.4.10

The screenshot shows the AWS Management Console 'Launch an instance' page. The 'Advanced network configuration' section is expanded, showing 'Network interface 1' with a 'Device index' of 0. The 'Subnet' is set to 'subnet-09d53b9b035621be1'. The 'Primary IP' is '172.31.7.255'. The 'Secondary IP' is '172.31.4.10'. The 'IPv6 Prefixes' section shows a warning: 'The selected subnet does not support IPv6 prefixes because it does not have an IPv6 CIDR.' The 'Interface type' is set to 'Select'. The 'Summary' section on the right shows 'Number of instances' as 1, 'Software Image (AMI)' as 'Canonical, Ubuntu, 26.04, amd64...', 'Virtual server type (instance type)' as 'c7i-flex.large', 'Firewall (security group)' as 'default', and 'Storage (volumes)' as '1 volume(s) - 25 GiB'. The 'Launch instance' button is visible.

Worker01 → 172.31.4.11

The screenshot shows the AWS Management Console 'Launch an instance' page. The 'Advanced network configuration' section is expanded, showing 'Network interface 1' with a 'Device index' of 0. The 'Subnet' is set to 'subnet-09d53b9b035621be1'. The 'Primary IP' is '172.31.7.255'. The 'Secondary IP' is '172.31.4.11'. The 'IPv6 Prefixes' section shows a warning: 'The selected subnet does not support IPv6 prefixes because it does not have an IPv6 CIDR.' The 'Interface type' is set to 'Select'. The 'Summary' section on the right shows 'Number of instances' as 1, 'Software Image (AMI)' as 'Canonical, Ubuntu, 26.04, amd64...', 'Virtual server type (instance type)' as 'c7i-flex.large', 'Firewall (security group)' as 'default', and 'Storage (volumes)' as '1 volume(s) - 25 GiB'. The 'Launch instance' button is visible.

Worker02 → 172.31.4.12

Yes. Based on your subnet 172.31.4.0/22, we'll use:

- **Master:** 172.31.4.10
- **Worker01:** 172.31.4.11
- **Worker02:** 172.31.4.12

Important: when launching the EC2 instances, manually assign these private IPs and make sure AWS shows them as available.

Also, I recommend **containerd as the Kubernetes runtime**. Docker Engine can be installed because your assignment asks for Docker, but Kubernetes itself will use containerd. Kubernetes documents that Docker Engine does not provide CRI directly, while containerd does.

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...
Worker-02	i-0298f02f22b6b07bd	Running	c7i-flex.large	3/3 checks passed	us-east-1a	ec2-100-54-43-45.com...	100.54.43.45
k8s-master	i-004f01f684ce82ec3	Running	c7i-flex.large	3/3 checks passed	us-east-1a	ec2-44-198-167-228.co...	44.198.167.228
Worker-01	i-0f618c342c64e66cd	Running	c7i-flex.large	3/3 checks passed	us-east-1a	ec2-3-239-168-126.co...	3.239.168.126

1. Master — Complete User Data

Paste this into **EC2 → Advanced details → User data** for the master:

```
#!/bin/bash
```

```
set -eux
```

```
# =====
```

1. SYSTEM UPDATE

=====

apt-get update -y

apt-get upgrade -y

=====

2. HOSTNAME

=====

hostnamectl set-hostname k8s-master

=====

3. HOSTS FILE

=====

cat <<EOF > /etc/hosts

127.0.0.1 localhost

127.0.1.1 k8s-master

172.31.4.10 k8s-master

172.31.4.11 worker01

172.31.4.12 worker02

EOF

=====

4. DISABLE SWAP

=====

swapoff -a

sed -i 's/^/#/' /etc/fstab

=====

5. KERNEL MODULES

=====

```
cat <<EOF > /etc/modules-load.d/k8s.conf
```

```
overlay
```

```
br_netfilter
```

```
EOF
```

```
modprobe overlay
```

```
modprobe br_netfilter
```

```
# =====
```

```
# 6. KERNEL PARAMETERS
```

```
# =====
```

```
cat <<EOF > /etc/sysctl.d/k8s.conf
```

```
net.bridge.bridge-nf-call-iptables = 1
```

```
net.bridge.bridge-nf-call-ip6tables = 1
```

```
net.ipv4.ip_forward = 1
```

```
EOF
```

```
sysctl --system
```

```
# =====
```

```
# 7. INSTALL REQUIRED PACKAGES
```

```
# =====
```

```
apt-get install -y \
```

```
    apt-transport-https \
```

```
    ca-certificates \
```

```
    curl \
```

```
    gpg
```

```
# =====
```

```
# 8. KUBERNETES REPOSITORY
```

```
# =====
```

```

mkdir -p -m 755 /etc/apt/keyrings

curl -fsSL \

    https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key \

    | gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
https://pkgs.k8s.io/core:/stable:/v1.29/deb/ /' \

    > /etc/apt/sources.list.d/kubernetes.list

# =====

# 9. INSTALL KUBERNETES

# =====

apt-get update -y

apt-get install -y kubelet kubeadm kubectl

apt-mark hold kubelet kubeadm kubectl

# =====

# 10. INSTALL CONTAINERD

# =====

apt-get install -y containerd

mkdir -p /etc/containerd

containerd config default > /etc/containerd/config.toml

# =====

# 11. CONFIGURE SYSTEMD CGROUP

# =====

sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' \

    /etc/containerd/config.toml

```

```

# =====

# 12. ENABLE CONTAINERD

# =====

systemctl restart containerd

systemctl enable containerd

# =====

# 13. INSTALL DOCKER

# =====

install -m 0755 -d /etc/apt/keyrings

curl -fsSL https://download.docker.com/linux/ubuntu/gpg \

    -o /etc/apt/keyrings/docker.asc

chmod a+r /etc/apt/keyrings/docker.asc

echo \

"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \

https://download.docker.com/linux/ubuntu \

$(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}") stable" \

> /etc/apt/sources.list.d/docker.list

apt-get update -y

apt-get install -y \

    docker-ce \

    docker-ce-cli \

    docker-buildx-plugin \

    docker-compose-plugin

systemctl enable docker

systemctl start docker

```

```
usermod -aG docker ubuntu
```

```
# =====
```

```
# 14. ENABLE KUBELET
```

```
# =====
```

```
systemctl enable kubelet
```

```
# =====
```

```
# 15. VERIFICATION
```

```
# =====
```

```
echo "===== MASTER SETUP COMPLETE ====="
```

```
hostname
```

```
docker --version
```

```
containerd --version
```

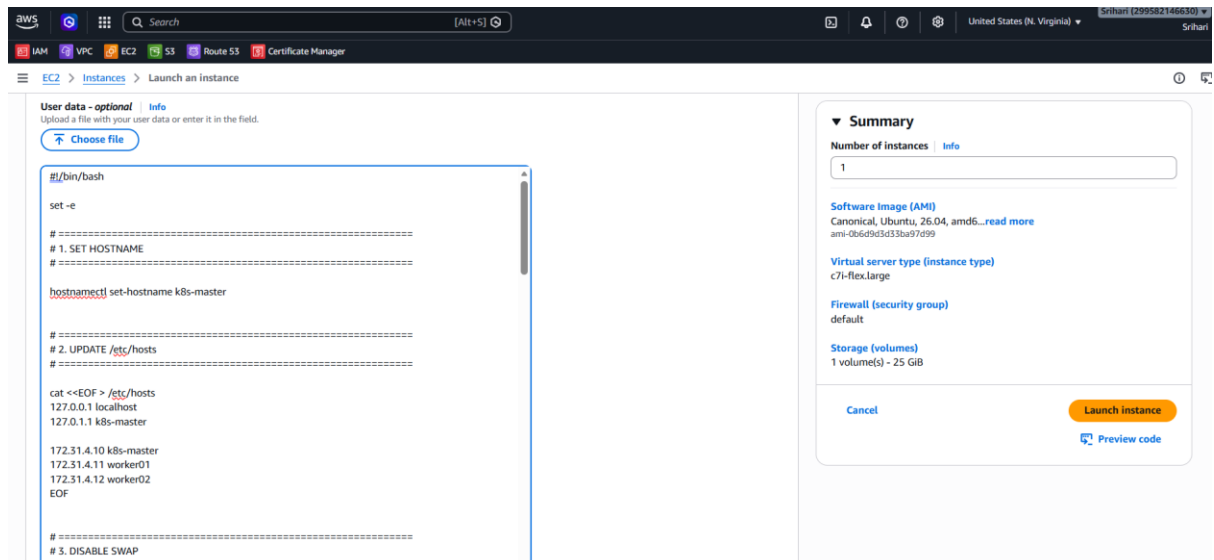
```
kubeadm version
```

```
kubectrl version --client
```

```
systemctl is-active containerd
```

```
systemctl is-active docker
```

```
echo "=====
```



Step 1 — Verify Master

Run these one by one:

hostname

Expected:

k8s-master

Then:

hostname -l

Expected to contain:

172.31.4.10

```
ubuntu@k8s-master:~$ hostname
k8s-master
ubuntu@k8s-master:~$ hostname -I
172.31.4.10 172.31.4.85
ubuntu@k8s-master:~$ sudo systemctl status containerd --no-pager
containerd.service - containerd container runtime
Loaded: loaded (/usr/lib/systemd/system/containerd.service; enabled; preset: enabled)
Active: inactive (dead) Since Tue 2026-09-22 06:50:16 UTC; 10min ago
Duration: 2.543s
Invocation: d9376b61af1f47b18ff4cea3clea87aa
Docs: https://containerd.io
Main PID: 7544 (code=exited, status=0/SUCCESS)
Mem peak: 16.9M
CPU: 53ms
Sep 22 06:50:14 k8s-master containerd[7544]: time="2026-09-22T06:50:14.162039444Z" level=info msg="Synchronizing NRI (plugin) with current runtime state"
Sep 22 06:50:14 k8s-master containerd[7544]: time="2026-09-22T06:50:14.162109836Z" level=info msg="containerd successfully booted in 0.018488s"
Sep 22 06:50:14 k8s-master systemd[1]: Started containerd.service - containerd container runtime.
Sep 22 06:50:16 k8s-master containerd[7544]: time="2026-09-22T06:50:16.705893068Z" level=info msg="Stop CRI service"
Sep 22 06:50:16 k8s-master systemd[1]: Stopping containerd.service - containerd container runtime...
Sep 22 06:50:16 k8s-master containerd[7544]: time="2026-09-22T06:50:16.708970247Z" level=info msg="Stop CRI service"
Sep 22 06:50:16 k8s-master containerd[7544]: time="2026-09-22T06:50:16.710029240Z" level=info msg="Event monitor stopped"
Sep 22 06:50:16 k8s-master containerd[7544]: time="2026-09-22T06:50:16.710034545Z" level=info msg="Stream server stopped"
Sep 22 06:50:16 k8s-master systemd[1]: containerd.service: Deactivated successfully.
Sep 22 06:50:16 k8s-master systemd[1]: Stopped containerd.service - containerd container runtime.
ubuntu@k8s-master:~$ kubectl version
Client version: v1.29.15
Kubernetes version: v1.29.15
ubuntu@k8s-master:~$ sudo kubectl init \
--apiserver-advertise-address=172.31.4.10 \
```

i-004f01f684ce82ec3 (k8s-master)
PublicIPs: 44.198.167.228 PrivateIPs: 172.31.4.85

Step 1 — Check containerd

Run:

```
sudo systemctl status containerd --no-pager
```

Then:

```
sudo systemctl is-active containerd
```

And:

```
ls -l /var/run/containerd/containerd.sock
```

Step 2 — Try restarting containerd

Run:

```
sudo systemctl restart containerd
```

Then:

```
sudo systemctl status containerd --no-pager
```

If it says **active (running)**, check:

```
ls -l /var/run/containerd/containerd.sock
```

You should see something like:

```
srw-rw---- ... /var/run/containerd/containerd.sock
```

Step 3 — Verify the SystemdCgroup setting

Run:

```
sudo grep SystemdCgroup /etc/containerd/config.toml
```

Expected:

```
SystemdCgroup = true
```

If it says false, run:

```
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
```

```
sudo systemctl restart containerd
```

Step 4 — Check containerd again

```
sudo systemctl is-active containerd
```

It should return:

active

Then check:

```
sudo crictl info
```

If crictl isn't installed, that's okay; we'll use another check.

Step 5 — Run kubeadm again

Only after containerd is active:

```
sudo kubeadm init \
  --apiserver-advertise-address=172.31.4.10 \
  --pod-network-cidr=10.244.0.0/16
```

Now we need to do **3 things on the Master**:

1. Configure kubectl
2. Install a Pod Network (CNI)
3. Join Worker01 and Worker02

Step 1 — Configure kubectl

You're logged in as ubuntu, so run:

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Then verify:

```
kubectl get nodes
```

You will probably see:

NAME	STATUS	ROLES	AGE	VERSION
k8s-master	NotReady	control-plane	...	v1.29.15

NotReady is expected right now because we haven't installed the Pod Network yet.

Step 2 — Install Pod Network

Because we initialized with:

```
--pod-network-cidr=10.244.0.0/16
```

we can use **Flannel**.

Run:

```
kubectl apply -f https://raw.githubusercontent.com/flannel-io/flannel/v0.25.5/Documentation/kube-flannel.yml
```

Then wait about 1–2 minutes and check:

kubectl get pods -A

You should start seeing Flannel pods running.

Then:

kubectl get nodes

The Master should eventually become:

k8s-master Ready control-plane ...

```
ubuntu@k8s-master:~$ mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
k8s-master    NotReady  control-plane  75s   v1.29.15

ubuntu@k8s-master:~$ kubectl apply -f https://raw.githubusercontent.com/flannel-io/flannel/v0.25.5/Documentation/kube-flannel.yml
namespace/kube-flannel created
clusterrole.rbac.authorization.k8s.io/flannel created
clusterrolebinding.rbac.authorization.k8s.io/flannel created
serviceaccount/flannel created
configmap/kube-flannel-cfg created
daemonset.apps/kube-flannel-ds created
ubuntu@k8s-master:~$ kubectl get pods -A
NAMESPACE     NAME                                READY   STATUS    RESTARTS   AGE
kube-flannel   kube-flannel-ds-frrk2              1/1     Running   0           61s
kube-system    coredns-76f75df574-tn4dj           1/1     Running   0           2m35s
kube-system    coredns-76f75df574-tqyb6           1/1     Running   0           2m35s
kube-system    etcd-k8s-master                    1/1     Running   0           2m52s
kube-system    kube-apiserver-k8s-master           1/1     Running   0           2m54s
kube-system    kube-controller-manager-k8s-master 1/1     Running   0           2m54s
kube-system    kube-proxy-4z5p5                   1/1     Running   0           2m36s
kube-system    kube-scheduler-k8s-master           1/1     Running   0           2m54s
ubuntu@k8s-master:~$ kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
k8s-master    Ready     control-plane  5m6s   v1.29.15
ubuntu@k8s-master:~$
```

i-004f01f684ce82ec3 (k8s-master)

PublicIPs: 44.198.167.228 PrivateIPs: 172.31.4.85

Step 3 — Join Worker01 and Worker02

Your kubeadm init generated this join command:

```
sudo kubeadm join 172.31.4.10:6443 --token smo0j7.ejvq0huapha67j3i \
```

```
--discovery-token-ca-cert-hash
```

```
sha256:52e6622570db025822a8ade13c793020a390ad05b31eb650fb98739e312d6472
```

Step 1 — Create Worker 01

In EC2 → **Launch Instance**:

- Same Ubuntu AMI as Master
- Same instance type
- VPC: vpc-00148488443198382
- Subnet: public-subnet
- **Private IP → Manually assign: 172.31.4.11**
- Auto-assign Public IP: **Enable**
- Security Group: **use the same security group as Master**

For User Data, use the Worker script. If you want, I can give you the **complete Worker-01 User Data** now.

Worker 01 — User Data

Paste this into Advanced details → User data:

```
#!/bin/bash

set -e

# =====
# 1. SET HOSTNAME
# =====

hostnamectl set-hostname worker01

# =====
# 2. UPDATE /etc/hosts
# =====

cat <<EOF > /etc/hosts
127.0.0.1 localhost
127.0.1.1 worker01

172.31.4.10 k8s-master
172.31.4.11 worker01
172.31.4.12 worker02
EOF

# =====
# 3. DISABLE SWAP
# =====

swapoff -a
sed -i 's/^/#/' /etc/fstab
```

```
# =====  
# 4. LOAD KUBERNETES KERNEL MODULES  
# =====
```

```
cat <<EOF > /etc/modules-load.d/k8s.conf  
overlay  
br_netfilter  
EOF
```

```
modprobe overlay  
modprobe br_netfilter
```

```
# =====  
# 5. KUBERNETES NETWORK SETTINGS  
# =====
```

```
cat <<EOF > /etc/sysctl.d/k8s.conf  
net.bridge.bridge-nf-call-iptables = 1  
net.bridge.bridge-nf-call-ip6tables = 1  
net.ipv4.ip_forward = 1  
EOF
```

```
sysctl --system
```

```
# =====  
# 6. UPDATE SYSTEM  
# =====
```

```
apt-get update -y
```

```
apt-get install -y \  
  apt-transport-https \  
  ca-certificates \  
  curl \  
  gpg
```

```
# =====  
# 7. INSTALL KUBERNETES REPOSITORY  
# =====
```

```
mkdir -p /etc/apt/keyrings
```

```
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key | \  
  gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
```

```
echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.29/deb/ \"" | \  
  tee /etc/apt/sources.list.d/kubernetes.list
```

```
# =====  
# 8. INSTALL KUBERNETES  
# =====
```

```
apt-get update -y
```

```
apt-get install -y kubelet kubeadm kubectl
```

```
apt-mark hold kubelet kubeadm kubectl
```

```
# =====
# 9. INSTALL CONTAINERD
# =====
```

```
apt-get install -y containerd
```

```
mkdir -p /etc/containerd
```

```
containerd config default > /etc/containerd/config.toml
```

```
sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' \
/etc/containerd/config.toml
```

```
systemctl restart containerd
systemctl enable containerd
```

```
# =====
# 10. INSTALL DOCKER
# =====
```

```
install -m 0755 -d /etc/apt/keyrings
```

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
-o /etc/apt/keyrings/docker.asc
```

```
chmod a+r /etc/apt/keyrings/docker.asc
```

```
echo \
"deb[arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
apt-get update -y
```

```
apt-get install -y \
docker-ce \
docker-ce-cli \
containerd.io \
docker-buildx-plugin \
docker-compose-plugin
```

```
systemctl enable docker
systemctl start docker
```

```
# =====
# 11. ENABLE KUBERNETES SERVICES
# =====
```

```
systemctl enable kubelet
systemctl start kubelet
```

```
# =====
# 12. VERIFICATION
# =====
```

```
echo "=====
echo "KUBERNETES WORKER PREPARATION COMPLETED"
```

```
echo "====="
```

```
echo "Hostname:"  
hostname
```

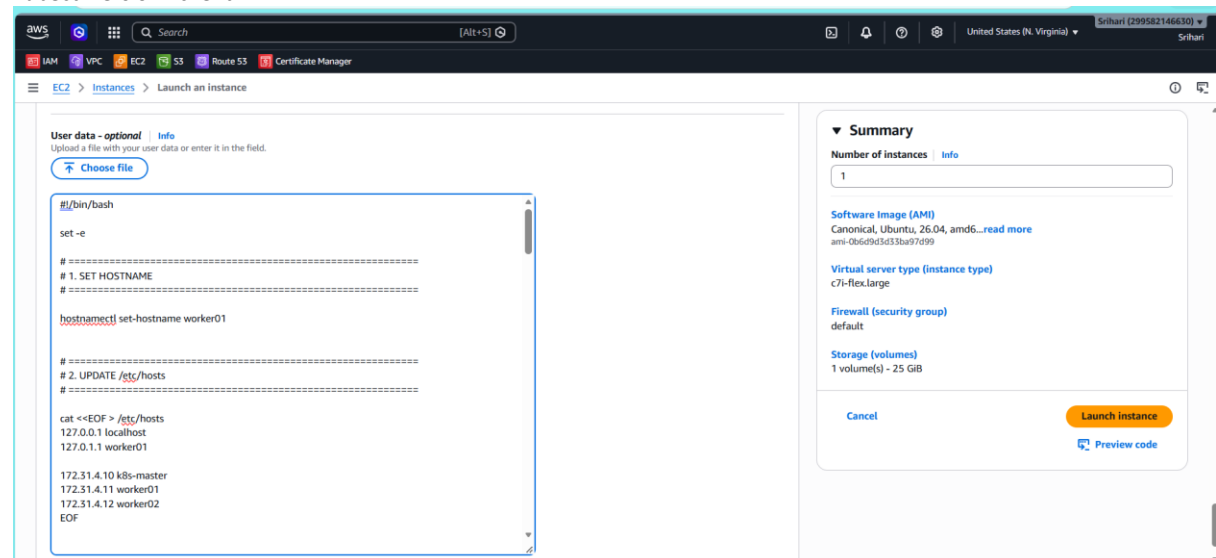
```
echo "Private IP:"  
hostname -I
```

```
echo "Containerd:"  
systemctl is-active containerd
```

```
echo "Docker:"  
systemctl is-active docker
```

```
echo "Kubelet:"  
systemctl is-active kubelet
```

```
echo "Kubernetes versions:"  
kubeadm version  
kubectl version --client
```



For Worker 02

Use **exactly the same script**, but change these two lines:

```
hostnamectl set-hostname worker02
```

and:

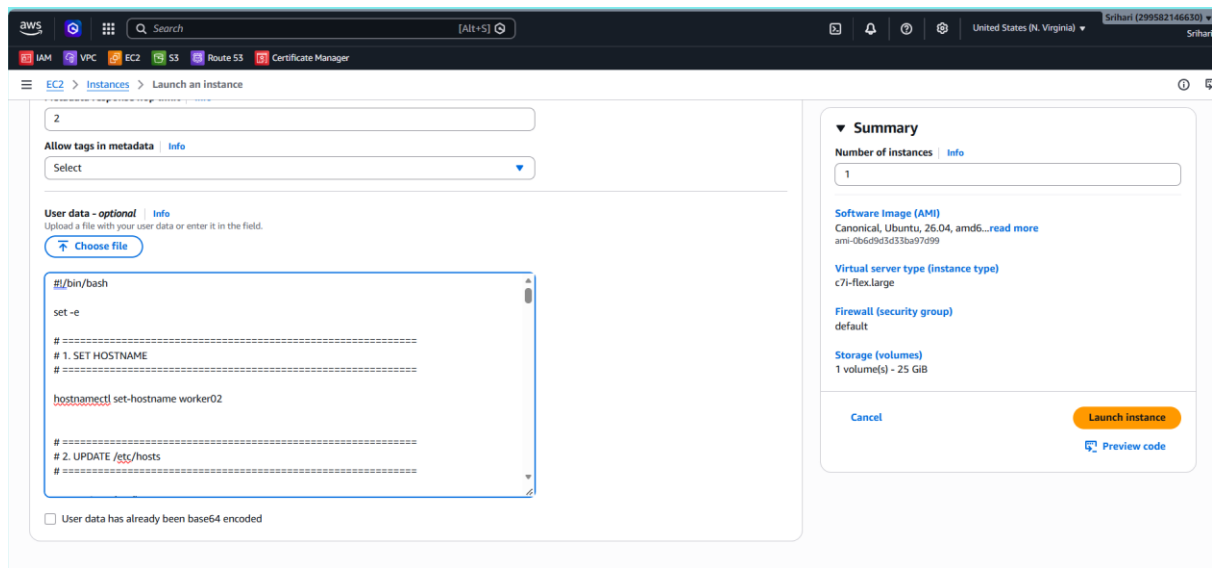
```
127.0.1.1 worker02
```

The /etc/hosts IP mapping **does not change**:

```
172.31.4.10 k8s-master
```

```
172.31.4.11 worker01
```

```
172.31.4.12 worker02
```



After both workers are created

SSH into each one and verify:

Worker 01

hostname

hostname -I

Expected:

worker01

172.31.4.11

Worker 02

hostname

hostname -I

Expected:

worker02

172.31.4.12

Step 1 — Start containerd on Worker01

Run:

```
sudo systemctl restart containerd
```

Then:

```
sudo systemctl status containerd --no-pager
```

You want:

Active: active (running)

```
abuntu@worker01:~$ hostname
worker01
abuntu@worker01:~$ sudo kubeadm join 172.31.4.10:6443 --token s=0j7.ejvq0huapha67j3i \
--discovery-token-ca-cert-hash sha256:52e6622570db025822a8ade13c793020a390ad05b31eb650fb98739e312d6472
[preflight] Running pre-flight checks
error execution phase preflight: [preflight] Some fatal errors occurred:
[ERROR CRI]: container runtime is not running: output: time="2026-09-22T07:15:02Z" level=fatal msg="validate service connection: validate CRI v1 runtime API for endpoint
ect: no such file or directory\""
error: exit status 1
[preflight] If you know what you are doing, you can make a check non-fatal with '--ignore-preflight-errors=...'
To see the stack trace of this error execute with --v=5 or higher
abuntu@worker01:~$ sudo systemctl restart containerd
abuntu@worker01:~$ sudo systemctl status containerd --no-pager
● containerd.service - containerd container runtime
   Loaded: loaded (/usr/lib/systemd/system/containerd.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-09-22 07:15:32 UTC; 7s ago
     Invocation: 6c9bf7397c00422694800a1b297c7a8
       Docs: https://containerd.io
    Process: 8522 ExecStartPre=/sbin/modprobe overlay (code=exited, status=0/SUCCESS)
   Main PID: 8523 (containerd)
      Tasks: 7
     Memory: 13M (peak: 14.7M)
        CPU: 43ms
    CGroup: /system.slice/containerd.service
            └─8523 /usr/bin/containerd
```

Step 2 — Check the socket

Run:

ls -l /run/containerd/containerd.sock

You should see the socket.

Then verify:

sudo grep SystemdCgroup /etc/containerd/config.toml

It should show:

SystemdCgroup = true

If it shows false

Run:

sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml

sudo systemctl restart containerd

Then verify again:

sudo systemctl is-active containerd

Expected:

Active

Step 3 — Join Worker01 again

Once containerd is active, run:

```
sudo kubeadm join 172.31.4.10:6443 --token smo0j7.ejvq0huapha67j3i \
```

```
--discovery-token-ca-cert-hash
```

```
sha256:52e6622570db025822a8ade13c793020a390ad05b31eb650fb98739e312d6472
```

```
ubuntu@worker01:~$ ls -l /run/containerd/containerd.sock
ls: cannot access '/run/containerd/containerd.sock': No such file or directory
ubuntu@worker01:~$ sudo grep SystemdCgroup /etc/containerd/config.toml
SystemdCgroup = true
ubuntu@worker01:~$ sudo kubeadm join 172.31.4.10:6443 --token smo0j7.ejvq0huapha67j3i \
--discovery-token-ca-cert-hash sha256:52e6622570db025822a8ade13c793020a390ad05b31eb650fb98739e312d6472
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system get cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap...

This node has joined the cluster:
* Certificate signing request was sent to apiservert and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.
ubuntu@worker01:~$
```

i-0f618c342c64e66cd (Worker-01)

PublicIPs: 3.239.168.126 PrivateIPs: 172.31.7.171

1. Connect to Worker02

SSH into Worker02 (172.31.4.12):

```
ssh ubuntu@172.31.4.12
```

Check:

hostname

Expected:

worker02

2. Start containerd

Because Worker02 may have the same User Data issue, run:

```
sudo systemctl restart containerd
```

Check:

```
sudo systemctl is-active containerd
```

Expected:

active

Check the socket:

```
ls -l /run/containerd/containerd.sock
```

Then:

```
sudo grep SystemdCgroup /etc/containerd/config.toml
```

Expected:

SystemdCgroup = true

If it is false:

```
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
```

```
sudo systemctl restart containerd
```

3. Join Worker02

Run:

```
sudo kubeadm join 172.31.4.10:6443 --token smo0j7.ejvq0huapha67j3i \
```

```
--discovery-token-ca-cert-hash
```

```
sha256:52e6622570db025822a8ade13c793020a390ad05b31eb650fb98739e312d6472
```

If successful, you'll see:

This node has joined the cluster

```
ubuntu@worker02:~$ hostname
worker02
ubuntu@worker02:~$ sudo systemctl restart containerd
ubuntu@worker02:~$ sudo systemctl is-active containerd
active
ubuntu@worker02:~$ ls -l /run/containerd/containerd.sock
srw-rw---- 1 root root 0 Sep 22 07:18 /run/containerd/containerd.sock
ubuntu@worker02:~$ sudo grep SystemdCgroup /etc/containerd/config.toml
    SystemdCgroup = true
ubuntu@worker02:~$ sudo kubeadm join 172.31.4.10:6443 --token smo0j7.ejvq0huapha67j3i \
    --discovery-token-ca-cert-hash sha256:52e6622570db025822a8ade13c793020a390ad05b31eb650fb98739e312d6472
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system get cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap...

This node has joined the cluster:
* Certificate signing request was sent to apiserver and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.
ubuntu@worker02:~$
```

```
i-0298f02f22b6b07bd (Worker-02)
PublicIPs: 100.54.43.45 PrivateIPs: 172.31.7.219
```

4. Check the cluster

Go back to **Master**:

```
kubectl get nodes -o wide
```

You should have:

NAME	STATUS	ROLES	INTERNAL-IP
k8s-master	Ready	control-plane	172.31.4.10
worker01	Ready	<none>	172.31.4.11
worker02	Ready	<none>	172.31.4.12

Last login: Tue Sep 22 06:57:42 2026 from 18.206.107.27

ubuntu@k8s-master:~\$ kubectl get nodes -o wide

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE	KERNEL-VERSION	CONTAINER-RUNTIME
k8s-master	Ready	control-plane	16m	v1.29.15	172.31.4.10	<none>	Ubuntu 26.04 LTS	7.0.0-1006-aws	containerd://2.3.5
worker01	Ready	<none>	4m19s	v1.29.15	172.31.4.11	<none>	Ubuntu 26.04 LTS	7.0.0-1006-aws	containerd://2.3.5
worker02	Ready	<none>	87s	v1.29.15	172.31.4.12	<none>	Ubuntu 26.04 LTS	7.0.0-1006-aws	containerd://2.3.5

ubuntu@k8s-master:~\$

i-004f01f684ce82ec3 (k8s-master)

PublicIPs: 44.198.167.228 PrivateIPs: 172.31.4.85

SWS

Q Search [Alt+S]

United States (N. Virginia)

Srihari (299582146630)

iam vpc ec2 s3 route 53 certificate manager

ubuntu@k8s-master:~\$ kubectl get nodes -o wide

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE	KERNEL-VERSION	CONTAINER-RUNTIME
k8s-master	Ready	control-plane	16m	v1.29.15	172.31.4.10	<none>	Ubuntu 26.04 LTS	7.0.0-1006-aws	containerd://2.3.5
worker01	Ready	<none>	4m19s	v1.29.15	172.31.4.11	<none>	Ubuntu 26.04 LTS	7.0.0-1006-aws	containerd://2.3.5
worker02	Ready	<none>	87s	v1.29.15	172.31.4.12	<none>	Ubuntu 26.04 LTS	7.0.0-1006-aws	containerd://2.3.5

ubuntu@k8s-master:~\$ kubectl get pods -A

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-flannel	kube-flannel-ds-42rhp	1/1	Running	0	5m21s
kube-flannel	kube-flannel-ds-frk2	1/1	Running	0	15m
kube-flannel	kube-flannel-ds-tzlkj	1/1	Running	0	2m29s
kube-system	coredns-76f75df574-tn4dj	1/1	Running	0	17m
kube-system	coredns-76f75df574-tqyb6	1/1	Running	0	17m
kube-system	etcd-k8s-master	1/1	Running	0	17m
kube-system	kube-apiserver-k8s-master	1/1	Running	0	17m
kube-system	kube-controller-manager-k8s-master	1/1	Running	0	17m
kube-system	kube-proxy-4z5p5	1/1	Running	0	17m
kube-system	kube-proxy-gz7hg	1/1	Running	0	2m29s
kube-system	kube-proxy-zhh6f	1/1	Running	0	5m21s
kube-system	kube-scheduler-k8s-master	1/1	Running	0	17m

ubuntu@k8s-master:~\$ kubectl get pods -A -o wide

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
kube-flannel	kube-flannel-ds-42rhp	1/1	Running	0	5m45s	172.31.4.11	worker01	<none>	<none>
kube-flannel	kube-flannel-ds-frk2	1/1	Running	0	16m	172.31.4.10	k8s-master	<none>	<none>
kube-flannel	kube-flannel-ds-tzlkj	1/1	Running	0	2m53s	172.31.4.12	worker02	<none>	<none>
kube-system	coredns-76f75df574-tn4dj	1/1	Running	0	17m	10.244.0.3	k8s-master	<none>	<none>
kube-system	coredns-76f75df574-tqyb6	1/1	Running	0	17m	10.244.0.2	k8s-master	<none>	<none>
kube-system	etcd-k8s-master	1/1	Running	0	18m	172.31.4.10	k8s-master	<none>	<none>
kube-system	kube-apiserver-k8s-master	1/1	Running	0	18m	172.31.4.10	k8s-master	<none>	<none>
kube-system	kube-controller-manager-k8s-master	1/1	Running	0	18m	172.31.4.10	k8s-master	<none>	<none>
kube-system	kube-proxy-4z5p5	1/1	Running	0	17m	172.31.4.10	k8s-master	<none>	<none>
kube-system	kube-proxy-gz7hg	1/1	Running	0	2m53s	172.31.4.12	worker02	<none>	<none>
kube-system	kube-proxy-zhh6f	1/1	Running	0	5m45s	172.31.4.11	worker01	<none>	<none>
kube-system	kube-scheduler-k8s-master	1/1	Running	0	18m	172.31.4.10	k8s-master	<none>	<none>

ubuntu@k8s-master:~\$ kubectl cluster-info

i-004f01f684ce82ec3 (k8s-master)

PublicIPs: 44.198.167.228 PrivateIPs: 172.31.4.85

3.Run one nginx pod.

1. Create one Nginx pod

kubectl run nginx --image=nginx

2. Check the pod

kubectl get pods

Expected:

NAME	READY	STATUS	RESTARTS	AGE
------	-------	--------	----------	-----

nginx	1/1	Running	0	10s
-------	-----	---------	---	-----

3. Check where the pod is running

kubectl get pod nginx -o wide

This will show whether Nginx is running on worker01 or worker02.

4. Test Nginx from the pod

kubectl exec -it nginx -- curl localhost

You should see the **Nginx welcome HTML**.

```

ubuntu@k8s-master:~$ curl localhost:30986
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
ubuntu@k8s-master:~$ kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
nginx     1/1     Running   0           3h32m
ubuntu@k8s-master:~$ kubectl get pod nginx -o wide

```

i-004f01f684ce82ec3 (k8s-master)

PublicIPs: 44.198.167.228 PrivateIPs: 172.31.4.85

Now do these 3 steps

1. On master, confirm NodePort:

```
kubectl get svc nginx
```

You should have:

80:30986/TCP

2. In AWS, get worker01's Public IPv4 address.

Go to:

EC2 → Instances → worker01 → Public IPv4 address

3. Add Security Group rule

For the Security Group attached to worker01:

Type: Custom TCP

Port: 30986

Source: My IP

Then open in your browser:

http://WORKER01-PUBLIC-IP:30986

For example:

http://54.xx.xx.xx:30986

You should see:

Welcome to nginx!



4. Mugup Master and slave components on k8s.

1. Control Plane (Master Node)

The control plane manages the entire Kubernetes cluster. It makes scheduling decisions and maintains the desired state.

Components:

1) kube-apiserver

- Entry point of the cluster
- All kubectl commands go through it
- Validates and processes API requests
- Communicates with etcd

2) etcd

- Key-value database
- Stores cluster state
- Stores:
 - Pods
 - Services
 - Secrets
 - ConfigMaps
 - Node information

3) kube-scheduler

- Assigns Pods to worker nodes
- Decides based on:
 - CPU and memory availability
 - Taints and tolerations
 - Node selectors and affinity rules

4) kube-controller-manager

Runs multiple controllers, such as:

- Node Controller
- ReplicaSet Controller
- Deployment Controller
- Job Controller

Ensures actual state matches desired state.

2. Worker Node (Slave Node)

Worker nodes run the application workloads.

Components:

1) kubelet

- Agent running on each worker node
- Receives instructions from control plane
- Ensures containers are running
- Reports node status to control plane

2) kube-proxy

- Handles networking rules
- Maintains iptables rules
- Enables communication between services and pods

3) Container Runtime

- containerd (recommended)
- Responsible for running containers
- Pulls images and starts containers

Architecture Flow

1. User runs kubectl command
2. Request goes to kube-apiserver
3. Scheduler selects a worker node
4. kubelet on that node creates the Pod
5. Container runtime runs the container

In Your Current Cluster

Master node:

- Runs control plane components:
 - kube-apiserver
 - etcd
 - scheduler
 - controller-manager

Worker nodes:

- Run:
 - kubelet
 - kube-proxy
 - containerd
- Host application Pods

